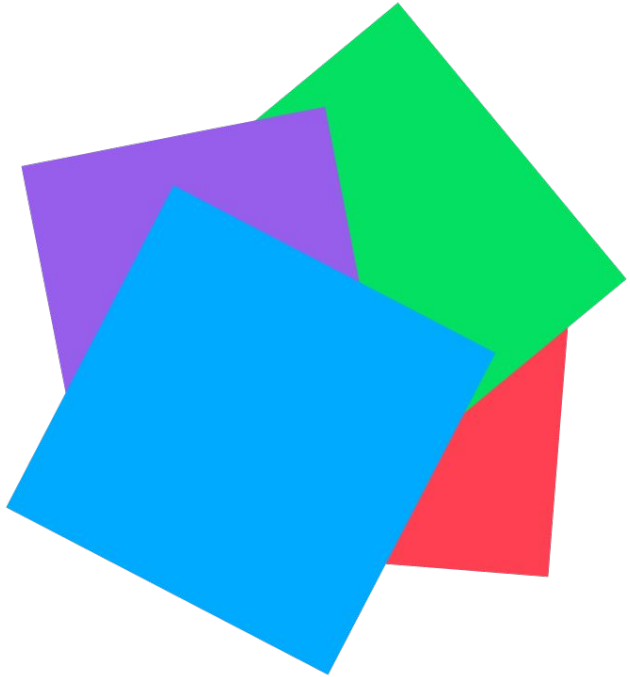
AI во фронтенде



Создаём планировщик учебных задач
и проверяем добавление задачи

Avito Start

Рассказывает Артем Кувалдин

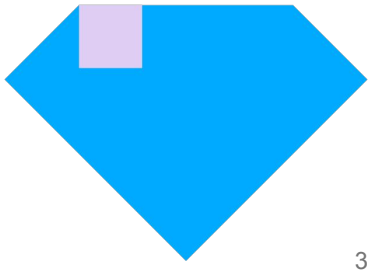
TUL / Talent Orchestration Platform

- Выпускник Мат-Мех УРФУ
- 14 лет в IT
- Опыт в Авито: разработка сервисов для сотрудников
- Люблю путешествия особенно в горы

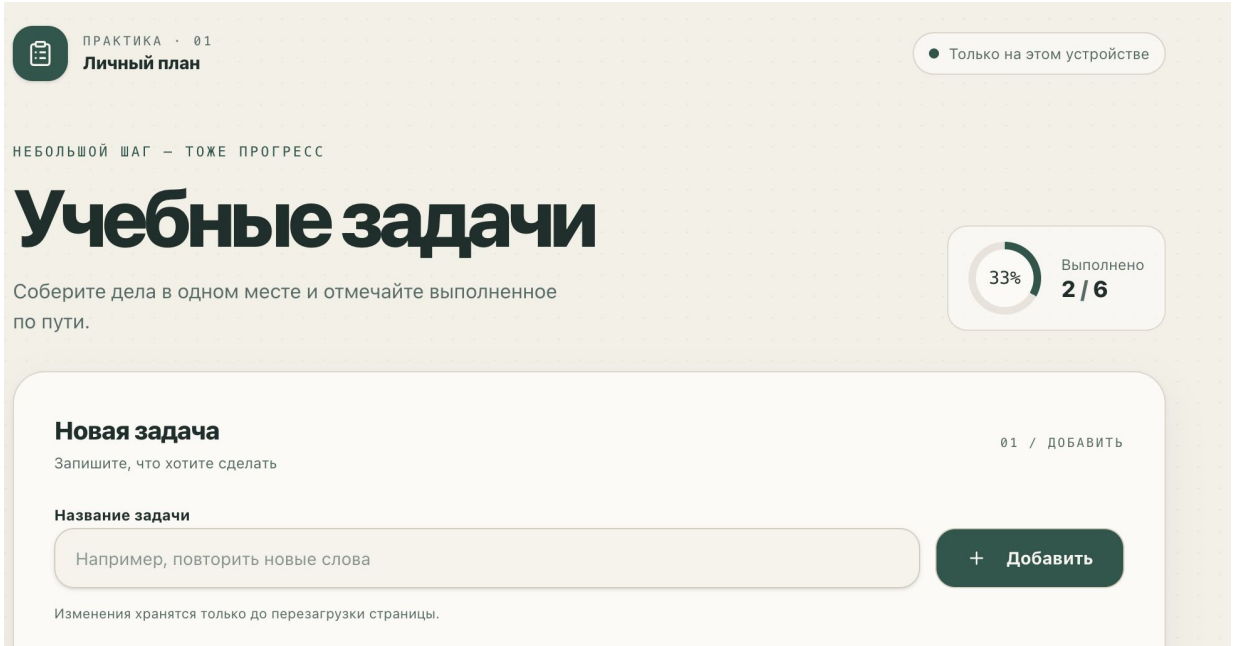


Avito Start

Как делать фронтенд с помощью AI?



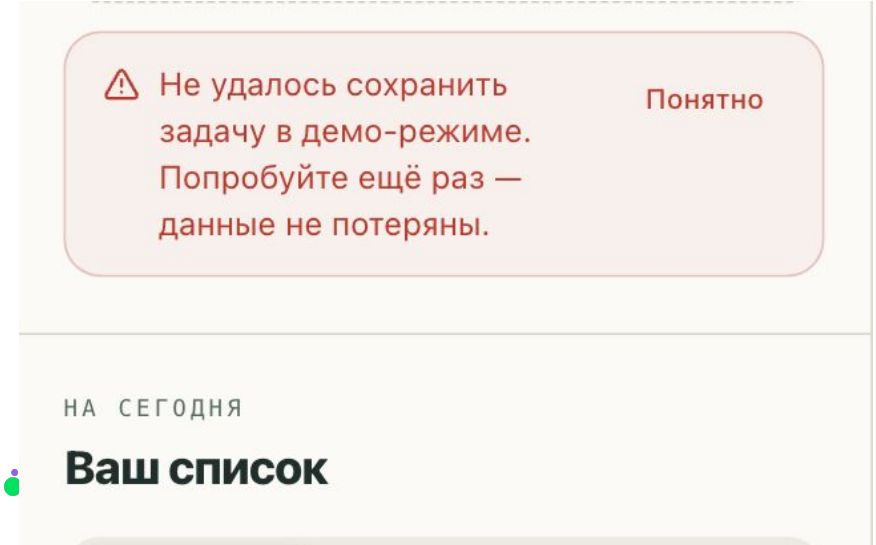
Представим, что агент уже создал такой экран. Готово ли приложение?



Внешний вид и поведение

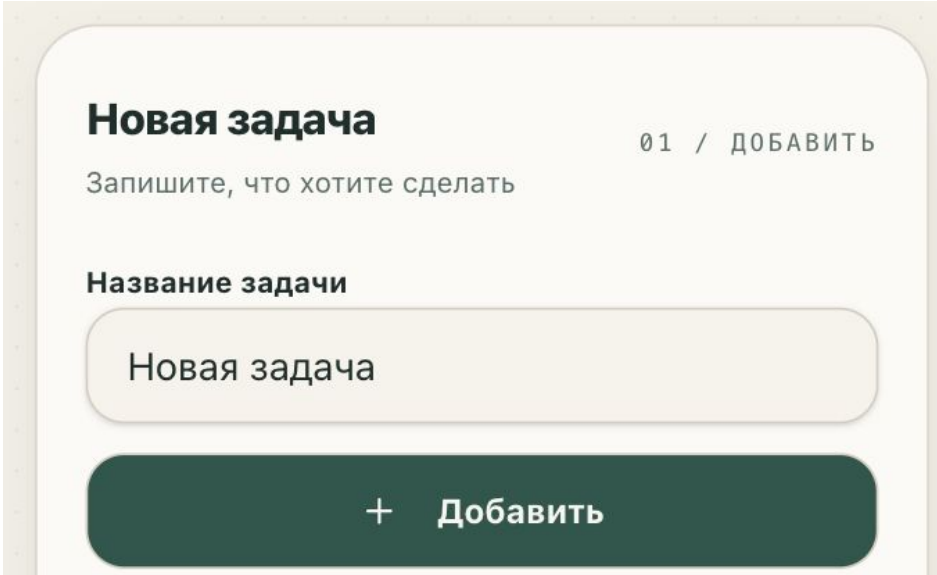
Поведение

Красивый экран но возникает ошибка с неработающей кнопкой

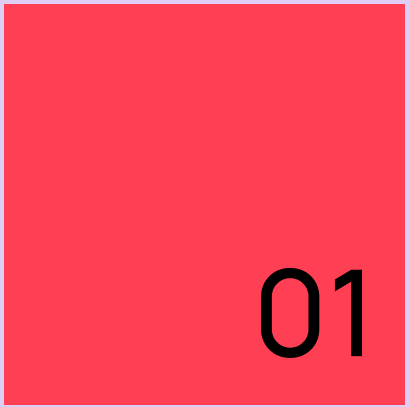


Внешний вид

Задача сохраняется, но отступы не те



Из чего состоит разработка фронтенда?



Этапы работы над приложением



Демо №2: запускаем скриншотные тесты Playwright и читаем отчёт

На чём строим фронтенд

Стек учебного планировщика задач

React + TypeScript

Компоненты, состояние
и типы данных интерфейса.

Vite

Локальный запуск приложения
и сборка для публикации.

Tailwind CSS

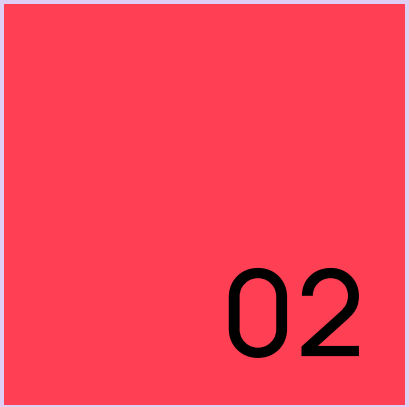
Стили интерфейса: цвета,
типографика и отступы.

shadcn/ui

Готовые компоненты в проекте:
кнопки, поля и диалоги.

Разработка — VS Code + Codex · Скриншотные тесты — Playwright

Как собрать макет?



Чем собрать первый вариант интерфейса



v0

Прототип для нашего демо



Figma Make

От дизайна к интерактивному прототипу



Lovable

От описания идеи к приложению



Replit Agent

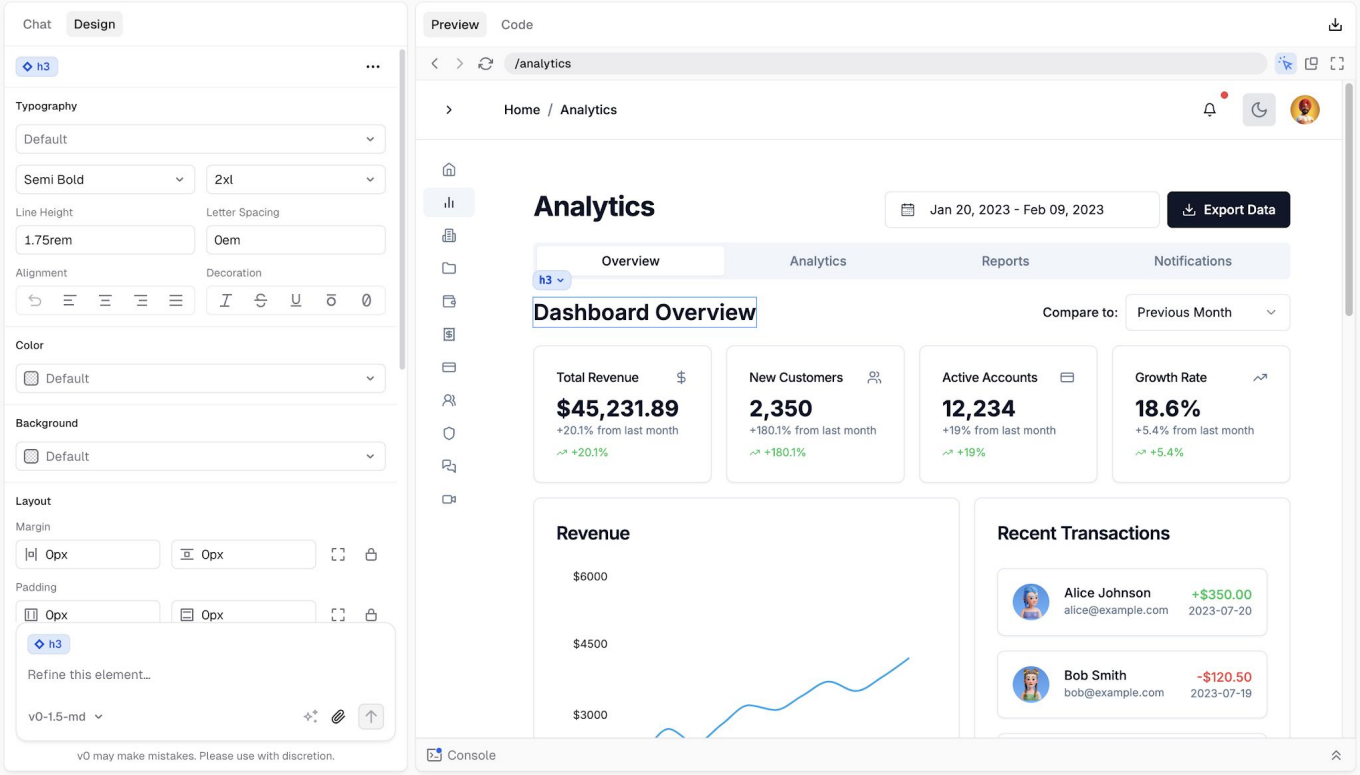
Создание и запуск в среде разработки

v0 (vercel): прототип с кодом

Описание интерфейса

Доработка: запрос или Design mode

Дальше: исходники и визуальный ориентир



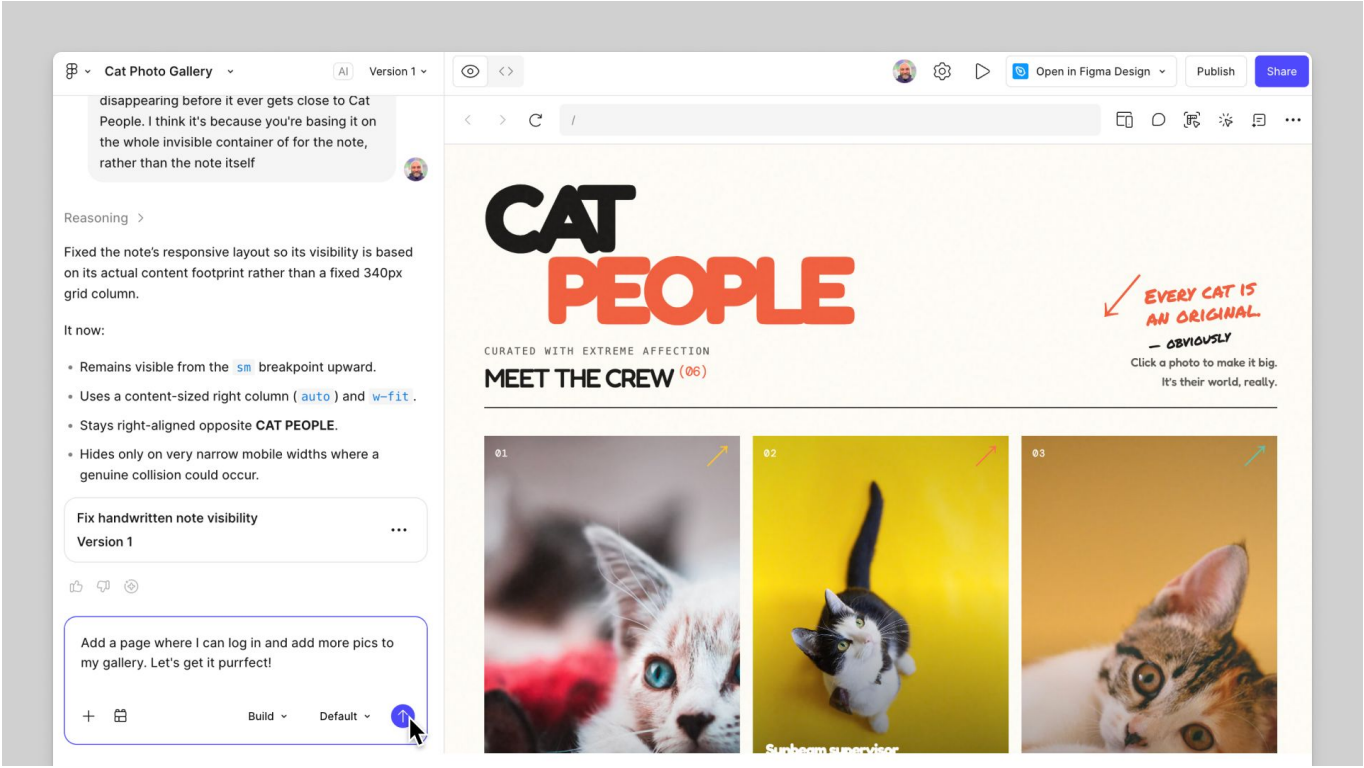
Официальный пример сервиса

Figma Make: интерактивный прототип

Дизайн и описание

Доработка: чат, выбор элемента и свойства

Дальше: прототип и состояния



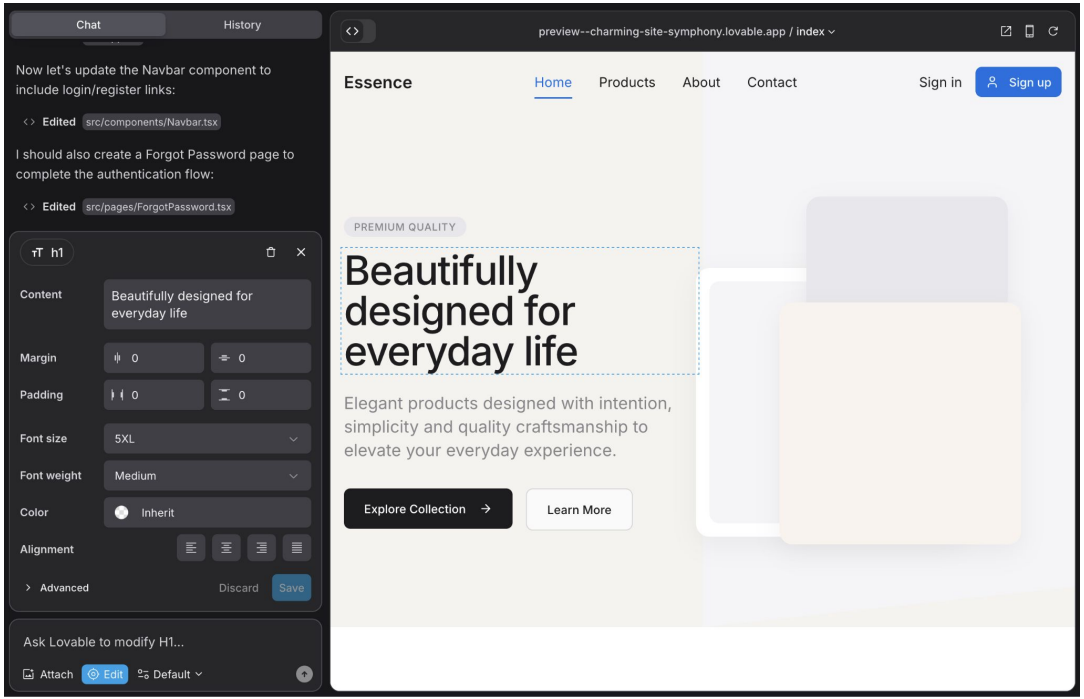
Официальный пример сервиса

Lovable: от идеи к приложению

Описание идеи

Доработка:
Select elements
и запрос; Edit
text inline

Дальше: проект
приложения



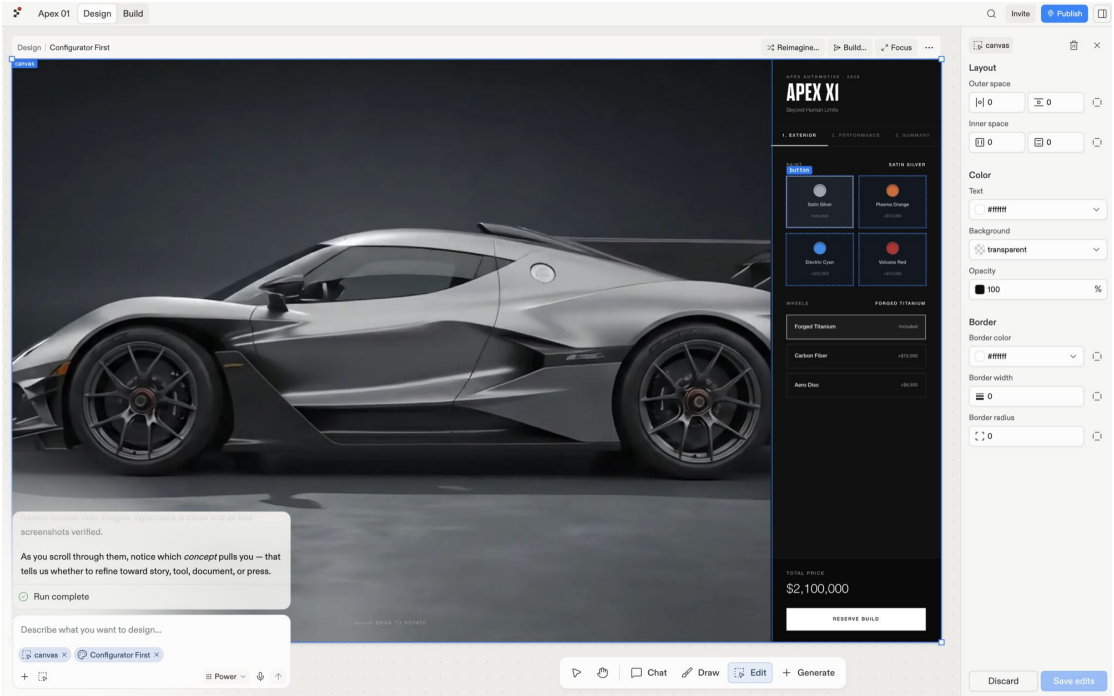
https://lovable.dev/blog/visual-edits

Replit Agent: создание и запуск

Приложение внутри среды разработки

Доработка: Agent или Visual Editor

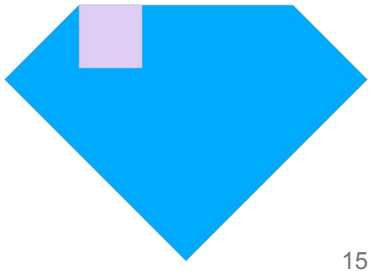
Дальше: код и запускаемый проект



Официальный пример Replit Design · Visual Editor

Replit Agent

для demo

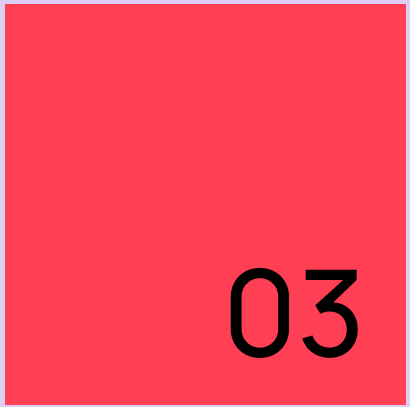


DEMO TIME

1. Обзор Replit
2. Создать прототип приложения
3. Попросить агента доработать прототип



Что делать с макетом?



Контекст для агента

Внешний вид

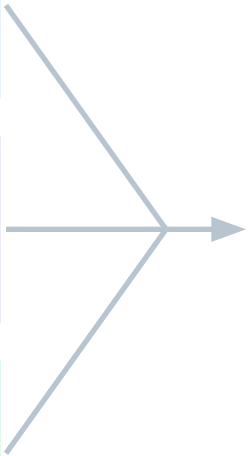
Макет и скриншоты

Поведение

Спецификация и состояния

Реализация

Компоненты и правила проекта



Задание агенту

Конкретное изменение и критерии приёмки

Чем точнее контекст, тем меньше догадок и переделок

Как передать макет

Передаём	Что получает агент	Чего не хватает
Скриншот	Видимое состояние экрана	Точной структуры и скрытых состояний
Структурированный макет	Компоненты, стили и размеры	Правил поведения и связи с проектом
Исходники прототипа	Основу вёрстки и логику прототипа	Проверки качества и совместимости
Ссылку на прототип	Доступные взаимодействия	Гарантированного доступа к коду

Скриншот и полный контекст

Только скриншот

МЕСТО ДЛЯ МАТЕРИАЛА
Первый фактический результат

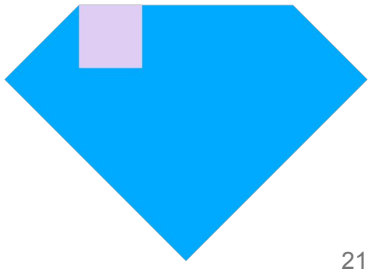
Полный контекст

МЕСТО ДЛЯ МАТЕРИАЛА
Второй фактический результат

Код + скриншот + требования
+ компоненты проекта

Материалы не представлены. Сравниваем поле, кнопку, компоновку и ошибку.

Какое поведение нельзя узнать из макета?



SDD: поведение задаёт спецификация

Spec-Driven Development:
разработка на основе согласованных требований

Макет задаёт внешний вид.
Спецификация описывает поведение.

Какое поведение нельзя узнать из макета?

Спецификация добавления задачи

`spec.md`

1. Название обязательно. Строка из пробелов не подходит.

2. Во время сохранения повторная отправка заблокирована.

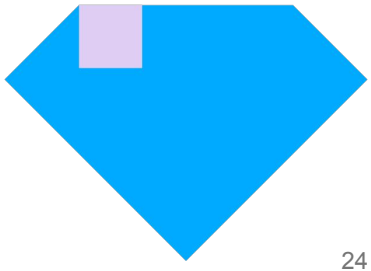
3. При успехе задача в списке, поле очищается.

4. При ошибке сохранения введённый текст сохраняется.

5. При ошибке рядом с полем показывается сообщение.

Приёмка после API 500 в поле остаётся точная введённая строка

Что делать если кнопки отличаются?



Популярные библиотеки компонентов

Три варианта для React-проекта

shadcn/ui

Исходники компонентов в вашем проекте

Удобно адаптировать под свой дизайн

Material UI

Готовые компоненты в стиле Material Design

Общая тема для всего интерфейса

Ant Design

Большой набор для бизнес-приложений

Формы, таблицы и панели управления

В нашем демо — shadcn/ui: адаптируем компоненты внутри проекта.

Компоненты в макете и коде

Кнопка макета → Button из проекта (shadcn/ui)

TaskForm.tsx / React + shadcn/ui

```tsx
import { Button } from '@/components/ui/button';

<Button type="submit" disabled={isSaving}>
  {isSaving ? 'Сохраняем…' : 'Добавить'}
</Button>
```

Учебный пример; пути и API сверить с демопроектом

Как агент использует компоненты

Компоненты + пример формы + правила
Задание на изменение → проверка использования

«Реализуй поведение формы по spec.md и выбранному макету.
Сначала назови компоненты, которые переиспользуешь».

AGENTS.md · инструкции проекта

```
Переиспользуй src/components/ui; изучи примеры и API.
Используй общие токены; не создавай дубликаты.
```

Инструменты для работы с проектом

Вариант	Роль в работе
Cursor Agent	Работа агента с проектом в редакторе
VS Code + Codex	Работа агента с проектом в VS Code
Claude Code	Работа агента с проектом через CLI

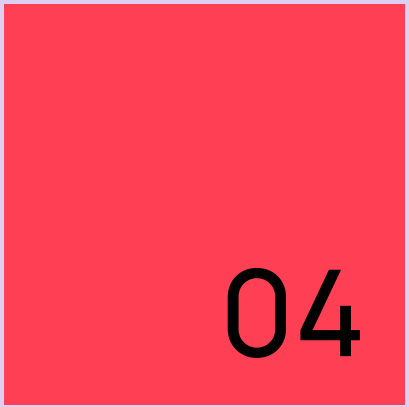
В демо: VS Code + расширение Codex от OpenAI

DEMO TIME

1. Берем макет
2. Передаем агенту
3. Просим доработать логику и использовать компоненты



Что если нашел баг?



Контекст ошибки

1

Ошибка вёрстки

Макет, скриншот, размер окна

2

Ошибка поведения

Шаги, ожидаемый и фактический результат

3

Ошибка API

Статус и тело ответа без секретов

Инструменты браузерной отладки

Инструмент	Для чего рассматриваем
Браузер Cursor	Быстрый просмотр результата в редакторе
Playwright MCP	Воспроизведение действий пользователя
Chrome DevTools MCP	Исследование Console, Network и производительности
Claude Code + Chrome	Работа с браузером через расширение

MCP — управление браузером агентом. Автотесты запускает Playwright Test.

Playwright MCP: браузер доступен агенту

Что может агент

Открывать страницы, нажимать кнопки и заполнять формы

Читать структуру интерфейса и снимать скриншоты

Чем это полезно

Воспроизвести проблему и проверить правку

Изучить реальный интерфейс перед подготовкой теста

«Открой планировщик, проверь мобильный вид и сделай снимок»

MCP помогает исследовать интерфейс; Playwright Test запускает сохранённые тесты.

Chrome DevTools MCP: диагностика

Что видит агент

Ошибки и сообщения консоли

Сетевые запросы и ответы

Данные о производительности

Чем это полезно

Найти причину сбоя

Разобраться в медленной загрузке

Проверить эффект исправления

«Проверь ошибку сохранения: изучи консоль и сетевой запрос»

Помогает найти причину проблемы; повторяемую проверку сохраняем в автотесте.

Настройка MCP для Codex

Node.js LTS и Chrome → конфигурация Codex → новый чат

~/.codex/config.toml

```
[mcp_servers.playwright]
command = "npx"
args = ["-y", "@playwright/mcp@latest"]

[mcp_servers.chrome-devtools]
command = "npx"
args = ["-y", "chrome-devtools-mcp@latest"]
```

Добавьте эти секции в config.toml, сохранив остальные настройки.

Проверяем подключение MCP

1. Откройте Codex

Откройте проект в VS Code
и панель Codex

Начните новый чат
Запустите приложение

2. Дайте агенту задачу

Playwright MCP:
«Открой localhost:3000
и сделай скриншот»

Chrome DevTools MCP:
«Открой localhost:3000
и покажи ошибки консоли»

Результат: агент вызывает MCP и возвращает снимок или данные консоли.

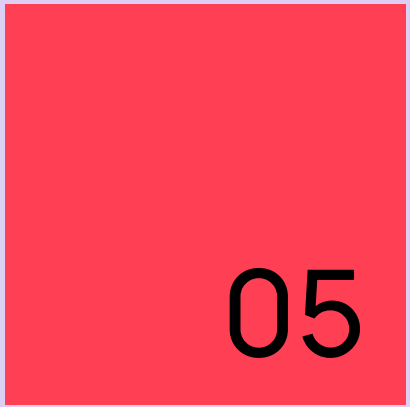
Укажите свой адрес приложения. Если подключения нет — проверьте Node.js и npx.

DEMO TIME

1. Использовать MCP Playwrite
2. Попросить прочитать что-то со страницы



Как тестировать?



Категории автотестов

Функции

Отдельная логика

Например: проверка названия задачи

Компоненты

Элемент интерфейса и взаимодействия

Например: форма показывает ошибку

Сценарии

Путь пользователя в браузере

Например: задача появилась в списке

Что проверяем: поведение · внешний вид (скриншоты) · доступность

Основные инструменты

Vitest	Запуск тестов функций и компонентов
React Testing Library	Проверка React-компонентов через действия
Playwright Test	Браузерные сценарии и сравнение скриншотов
axe-core	Автоматические проверки доступности

Для демо используем Playwright Test

Как работает скриншотный тест

1. Эталон

Согласованный снимок интерфейса

2. Новый снимок

То же состояние после изменения

3. Сравнение

Playwright показывает визуальные различия

```
await expect(page).toHaveScreenshot('planner.png');
```

Эталон — снимок приложения, проверенный человеком; не файл макета

Снимки должны быть воспроизводимыми

Одинаковое окружение

Размер окна и браузер

ОС, шрифты и масштаб

Отдельные эталоны
для разных конфигураций

Одинаковое состояние

Фиксированные тестовые данные

Дождаться загрузки интерфейса

Убрать анимации;
стабилизировать
время и случайные значения

Иначе тест будет находить случайные различия вместо ошибок интерфейса

Демо: запускаем скриншотные тесты

Запуск → отчёт → разбор различий → решение

Терминал / Playwright Test

```
npx playwright test tests/visual.spec.ts

npx playwright show-report
```

Смотрим, что изменилось — и должно ли было измениться

Демо: читаем отчёт Playwright

Expected

Одобренный эталон

Место для
изображения
из реального отчёта

Actual

Текущий снимок

Место для
изображения
из реального отчёта

Diff

Область различий

Место для
изображения
из реального отчёта

Резервный слайд. До лекции заменить подписи изображениями одного сравнения.

Различие найдено: что делать?

Неожиданное изменение

Исправить приложение

Повторить запуск

Проверить отчёт

Ожидаемое изменение

Проверить новый внешний вид

Одобрить его

Обновить эталон и запустить снова

Не обновляем эталоны автоматически только ради зелёного теста

Обновляем эталон после проверки

```
npx playwright test tests/visual.spec.ts \
  --update-snapshots

npx playwright test tests/visual.spec.ts
```

Просмотреть изменения снимков и сохранить их в Git вместе с тестом и изменением интерфейса.

Резервный слайд. Обновление эталона — осознанное решение.

Что скриншотные тесты не заменяют

Поведение

Клики, ввод
и сохранение

Проверяем действия
и результат

Доступность

Подписи,
клавиатура,
фокус и контраст

axe-core + ручная
проверка

Оценка дизайна

Удобство и польза

Эталон проверяет
человек

Скриншотный тест отвечает на вопрос: «Внешний вид изменился?»

DEMO TIME

1. Написать скриншотный тест
2. Сломать и посмотреть отчет



Весь процесс: внешний вид и поведение

Прототип и уточнение
Выбранный внешний вид

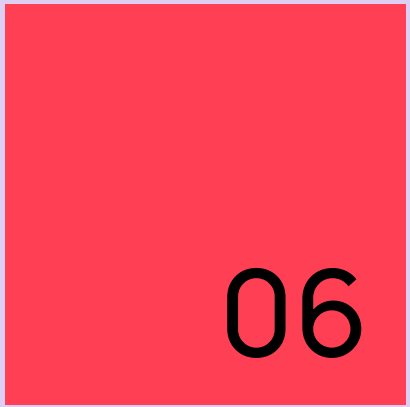
Спецификация
Ожидаемое поведение

Компоненты и правила
Способ реализации

Агент вносит изменения

Разработчик проверяет поведение и внешний вид

Итого



Итог

1

Задайте контекст

Макет, требования, дизайн-система

2

Соберите интерфейс

Один полный пользовательский сценарий

3

Проверьте результат

Поведение, внешний вид и доступность

AI ускоряет работу.
Качество проекта подтверждают работающий сценарий и проверки.

Важное для экзамена

07

Критерии хорошего фронтенд-проекта

Макет с AI

- Создан или доработан макет в выбранном инструменте.

Библиотека компонентов

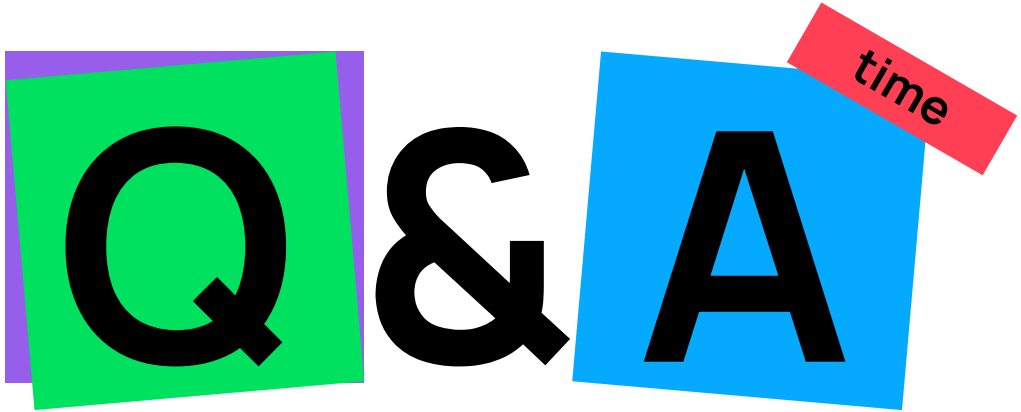
- AI использует кнопки, поля, карточки выбранной дизайн-системы.
- Единые цвета, шрифты, отступы.

Фронтенд и интерфейс

- Полный пользовательский сценарий.
- Соответствие макету или прототипу.
- Подписи полей, клавиатура, видимый фокус.

Тестирование с AI
Все тесты проходят

- Защита формы от случайной повторной отправки.
- Отображение компонентов; кнопки, ввод, открытие и закрытие диалогов.
- Валидация: обязательные поля, неверный ввод, сообщения об ошибках.
- Загрузка, пустые данные, успех, ошибка. Ответы сервера подменены.
- Тест основного сценария на клиенте: от действия до видимого результата.


Q&A
time